



学会使用Buffer的相关操作

北京理工大学计算机学院
金旭亮

clear与flip操作



`clear()`

还原缓冲区到初始的状态，将`position`设置为0，将`limit`设置为容量，这个通常用于准备重新开始写入数据。

`flip()`

反转此缓冲区。首先将`limit`设置为当前位置，然后将`position`设置为0，这个通常用于读取前面已经写入的数据。



先向缓冲区中存储数据，然后再从缓冲区中读取已经写入的这些数据，这是使用`flip()`方法的最佳时机。

理解flip与clear操作

```
String[] poem =
{
    "海上生明月，天涯共此时。",
    "情人怨遥夜，竟夕起相思。",
    "灭烛怜光满，披衣觉露滋。",
    "不堪盈手赠，还寝梦佳期"
};

CharBuffer buffer = CharBuffer.allocate(250);
//向缓冲区中写入数据
for (String s : poem) {
    for (int j = 0; j < s.length(); j++)
        buffer.put(s.charAt(j));
}
```

```
System.out.println("向缓冲区中写入数据之后");
MyBufferUtils.printBufferInfo(buffer);
//翻转缓冲区
buffer.flip();
System.out.println("翻转缓冲区之后");
MyBufferUtils.printBufferInfo(buffer);
// 读出数据
while (buffer.hasRemaining())
    System.out.print(buffer.get());
//清空缓冲区
buffer.clear();
System.out.println("清空缓冲区之后");
MyBufferUtils.printBufferInfo(buffer);
System.out.println();
```

FlipAndClear.java

运行截图

向缓冲区中写入数据之后

```
=====
Capacity: 250
Limit: 250
Position: 47
Remaining: 203
=====
```

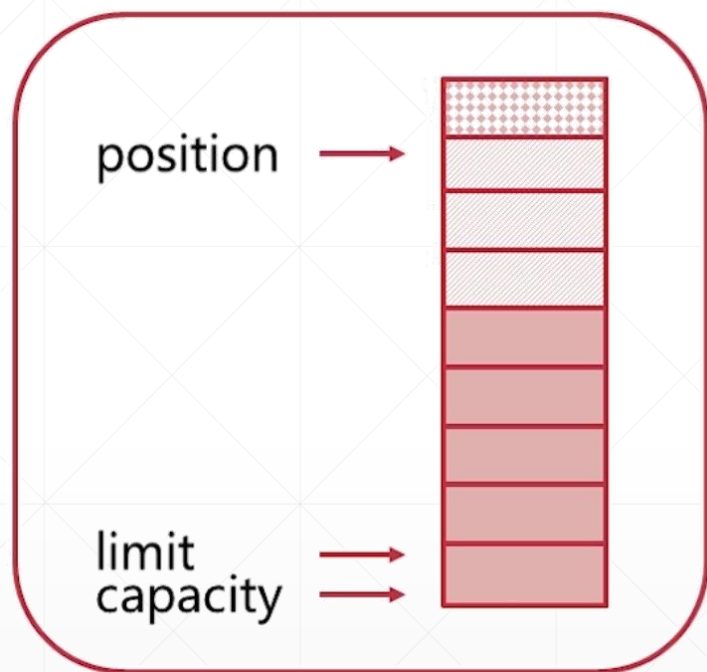
翻转缓冲区之后

```
=====
Capacity: 250
Limit: 47
Position: 0
Remaining: 47
=====
```

清空缓冲区之后

```
=====
Capacity: 250
Limit: 250
Position: 0
Remaining: 250
=====
```

理解compact操作



将已经读过的内容移除，然后将未读的内容移到缓冲区前端，`position`指向复制的所有数据之后的“下一个位置”，这样，就立即可以调用`put`方法写入数据了。

Compact 示例

```
private static void testCompact(){
    char[] charArr = {'0','1','2','3','4','5','6','7','8','9'};
    CharBuffer charBuffer = CharBuffer.wrap(charArr);
    //0 1 2 3 4 5 6 7 8 9
    while (charBuffer.hasRemaining()){
        System.out.print(charBuffer.get()+" ");
    }
    System.out.println("\n设置position=2");
    charBuffer.position(2);
    MyBufferUtils.printBufferInfo(charBuffer);
    System.out.println("执行compact操作");
    charBuffer.compact();
    MyBufferUtils.printBufferInfo(charBuffer);
    System.out.println("输出内容");
    //8,9
    while (charBuffer.hasRemaining()){
        System.out.print(charBuffer.get()+" ");
    }
}
```

Run: UseBuffer x

"C:\Program Files\Java\jdk-17\bin\java.exe"

0 1 2 3 4 5 6 7 8 9

设置position=2

=====

Capacity: 10

Limit: 10

Position: 2

Remaining: 8

=====

执行compact操作

=====

Capacity: 10

Limit: 10

Position: 8

Remaining: 2

=====

输出内容

8 9

Process finished with exit code 0

书签功能示例



```
public class UseBufferMark {  
    public static void main(String[] args) {  
        ByteBuffer buffer = ByteBuffer.allocate(7);  
        buffer.put((byte) 10).put((byte) 20).put((byte) 30).put((byte) 40);  
        buffer.limit(4);  
        //给第2个位置加上一个书签，之后再移动到第4个位置  
        buffer.position(1).mark().position(3);  
        System.out.println(buffer.get()); //40  
        System.out.println();  
        //回到书签处  
        buffer.reset();  
        //输出剩余的内容： 20,30,40,  
        while (buffer.hasRemaining())  
            System.out.print(buffer.get()+",");  
        System.out.println();  
        //回到开头： mark清除，位置position值归0， limit不变。  
        buffer.rewind();  
        MyBufferUtils.printBufferInfo(buffer);  
    }  
}
```

书签功能用于“记住”一个特殊的位置，之后可以回到这个位置，读写数据。

```
Run UseBufferMark x  
"C:\Program Files\Java\jdk-20\bin\java.exe"  
40  
20,  
30,  
40,  
=====  
Capacity: 7  
Limit: 4  
Position: 0  
Remaining: 4  
=====  
Process finished with exit code 0
```


mark详解



mark是一个索引，在调用reset()方法时，会将缓冲区的 position位置重置为该索引。



定义mark时，不能将其定义为负数，并且不能让它大于position。



如果定义了mark，则在将position或limit调整为小于该mark的值时，该mark被丢弃，丢弃后mark的值是-1。



如果未定义mark，那么调用reset()方法将导致抛出InvalidMarkException异常。